

From Protocol Semantics to Executive Code: Agentic LLM Planning for Network Protocol Implementation

Anonymous submission

Abstract

Technical documents, including RFCs and developer manuals, are primary knowledge sources for network protocol implementation. Prior work have explored extracting structured protocol facts that capture high-level protocol semantics from technical documents, as well as constraining LLM-based code generation with implementation-oriented specifications. However, the former omits module, interface, and dependency design decisions, while the latter assumes these decisions have been made. This paper bridges the problem from protocol facts to code generation as an intermediate planning task that transforms protocol knowledge into implementation-oriented specifications. To this end, we propose SpecForge, a modular planning framework that utilizes LLM-based derivation to construct protocol-aware specifications capturing engineering structure and protocol semantics. On MQTT, SpecForge reduces pre-repair root-cause defect groups by 61.9% and 73.9% compared with LLM-based direct generation and natural-language planning. Across HTTP/1.1, MQTT, CoAP, and SMTP, SpecForge improve the behavioral test pass rate from 70.3% to 94.8%. These results validate the feasibility of modular implementation-oriented specification planning and protocol-aware specification improves code buildability and behavioral correctness.

1 Introduction

Network protocols are among the most fundamental software components of Internet systems, and they are specified described in technical documents such as RFCs, standards, and developer manuals. These documents contain extensive *Protocol Facts*, including message formats, state transitions, and interaction patterns. Nevertheless, implementing a complete protocol from these documents remains a challenging task that relies heavily on human expertise. Developers have to not only understand the protocol facts but also translate them into concrete *Implementation-oriented Specifications*, including module decomposition, API design, data structure definitions, and dependency management.

LLMs have been widely applied to code generation, program repair, and software engineering automation, making

it possible to assist the generation of protocol implementations from protocol documents. However, directly applying LLM to generate a complete protocol implementation from an RFC or developer manual is generally unreliable (Long, Su, and Han 2026). Protocol documents specify externally observable semantics but rarely explain how these semantics should be mapped to concrete software structures. For example, a document may define how a field is encoded without specifying which modules should be responsible for parsing it. Therefore, the most critical challenge in protocol implementation generation is transforming high-level protocol facts into implementation-oriented specifications.

Existing research has mainly proceeded along two directions. One line of work extracts state machines, message constraints, and structured facts from protocol documents (Sharma and Yegneswaran 2023; Ishtiaq et al. 2024; Ma, Luo, and Zeng 2024). The other generates protocol-testing artifacts, protocol code, or system implementations under the assumption that specifications are already available (Liu et al. 2026; Wang et al. 2022). However, a critical gap remains between these two directions. Current document-understanding approaches largely remain at the level of high-level semantic extraction, and their outputs cannot directly drive engineering implementation. Conversely, code-generation approaches typically assume that engineering specifications have already been provided by developers. Bridging this gap is nontrivial. We summarize the challenges from protocol facts into implementation-oriented specifications as follows:

Challenge I: Unclear Task Definition of Protocol Implementation Planning. Protocol implementation is complex because code for protocol behaviors spans interdependent modules and must coordinate through interfaces and processing flows. Protocol implementation planning transforms protocol facts and target implementation constraints into implementation-oriented specifications of software structure and behavior. It begins after fact extraction and ends before code generation. Document-understanding methods extract protocol facts (Sharma and Yegneswaran 2023; Ishtiaq et al. 2024; Ma, Luo, and Zeng 2024), whereas specification-driven code-generation methods consume implementation-oriented specifications (Liu et al. 2026). However, neither line of work explicitly defines this intermediate transformation. The challenge is therefore to formulate a bounded planning task that converts protocol facts into implementation-

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

oriented specifications.

Challenge II: Inconsistent Specification Derivation.

Protocol documents do not directly prescribe module responsibilities or interface designs. Therefore, protocol facts do not have a unique or direct mapping to engineering specifications. The transformation requires evidence-constrained engineering choices among multiple feasible designs, while maintaining consistency throughout the derivation process. For example, a CoAP document specifies ACK timeout and retransmission rules. If responsibilities are assigned inconsistently, the subsequently derived callback interfaces and call paths may conflict, potentially causing retransmission to continue even after an ACK has been received. The key difficulty is to progressively resolve an unclear protocol description into a complete and consistent engineering design.

Challenge III: Insufficient Protocol Semantics in General-Purpose Specifications.

SYSSPEC (Liu et al. 2026) demonstrates that functional, module, and concurrency specifications can provide LLMs with a structured blueprint for code generation and support the generation of complex systems. However, general-purpose code specifications cannot be directly applied to the network protocol domain. Consider an SMTP session: a general-purpose specification may separately describe the MAIL, RCPT, and DATA functions, but it is difficult to constrain the transactional state spanning these commands. MAIL requires prior greeting and authentication, RCPT depends on the sender being established, and DATA depends on at least one recipient being available. Without unified constraints over commands, session states, and processing flows, individually correct functions may still compose into invalid interactions.

To address these challenges, we propose SpecForge, an implementation-oriented specification planning framework consisting of three procedures: protocol fact extraction, implementation-oriented planning, and specification-driven code generation. To handle the unclear mapping from protocol facts to implementation-oriented specifications, SpecForge proposes a modular planning mechanism in which the LLM derives only localized engineering decisions from protocol facts, while shared specification representations and cross-stage consistency constrain the overall derivation. Secondly, SpecForge further augments general-purpose software specifications with protocol-specific semantic dimensions that explicitly capture the information required for protocol implementation. Its core innovation is a verifiable planning layer that guides LLM-driven reasoning and code generation by unifying protocol logic, engineering design, and consistency enforcement into a coherent blueprint for code generation. On MQTT, Full-SpecForge reduces pre-repair root-cause defect groups by 61.9% and 73.9% compared with LLM-based direct generation and natural-language planning, respectively, while improving the structural and semantic quality of generated projects. Separately, across HTTP/1.1, MQTT, CoAP, and SMTP, all 40 projects using manually constructed protocol-aware specifications compile within bounded repair and pass 275 of 290 behavioral tests, increasing the pass rate from 70.3% to 94.8%. These results show that modular planning improves the structural and semantic quality of generated code, while protocol-aware specification

content supports downstream buildability and behavioral correctness.

We summarize our contributions as follows:

- (1) **We propose and formalize implementation-oriented specification planning as an intermediate task between protocol fact extraction and specification-guided code generation.** The task transforms evidence-grounded protocol facts and target constraints into specifications appropriate for downstream code generation.
- (2) **We design SpecForge, a modular framework for constructing protocol-aware specifications.** Through structured intermediate artifacts and validator-enforced constraints, SpecForge transforms evidence-grounded protocol facts into engineering specifications for downstream code generation.
- (3) **We extend protocol specifications with structural and behavioral constraints and evaluate them on four protocols.** All generated multi-module C projects compile within bounded repair and pass 94.8% of behavioral tests.

2 Background & Motivation

2.1 From Technical Documents to Protocol Facts

Prior studies have attempted to transform protocol documents into structured protocol facts. For example, research on finite-state machine extraction recovers state machines from protocol specifications and further applies them to attack synthesis or protocol analysis (Pacheco et al. 2022). PROSPER employs large language models to extract protocol automata from RFC text and diagrams (Sharma and Yegneswaran 2023). LLM-guided protocol fuzzing uses LLMs to derive message formats, message sequences, and state-exploration information from protocol documents (Meng et al. 2024). PARVAL represents protocol standards and parser implementations using a unified format specification to identify inconsistencies between implementations and specifications (Zheng, Xie, and Zhang 2025). These studies demonstrate that factual knowledge embedded in protocol documents can be made explicit and used for analysis, verification, testing, and fuzzing. However, such structured facts do not directly determine how a protocol implementation should be constructed. State machines, message formats, and field constraints describe what a protocol is, but do not address the concrete engineering decisions required for its implementation.

2.2 From Specifications to Code Generation

Another line of work investigates how specifications guide code generation. LLM agents incorporate planning, tool use, feedback, and repair (Jiang et al. 2026; Jin et al. 2024), while CodePlan plans interdependent repository-wide edits (Bairi et al. 2024). MetaGPT and ChatDev organize design, coding, and testing into staged multi-agent workflows (Hong et al. 2024; Qian et al. 2024). These approaches target general software development rather than deriving implementation architectures from protocol knowledge.

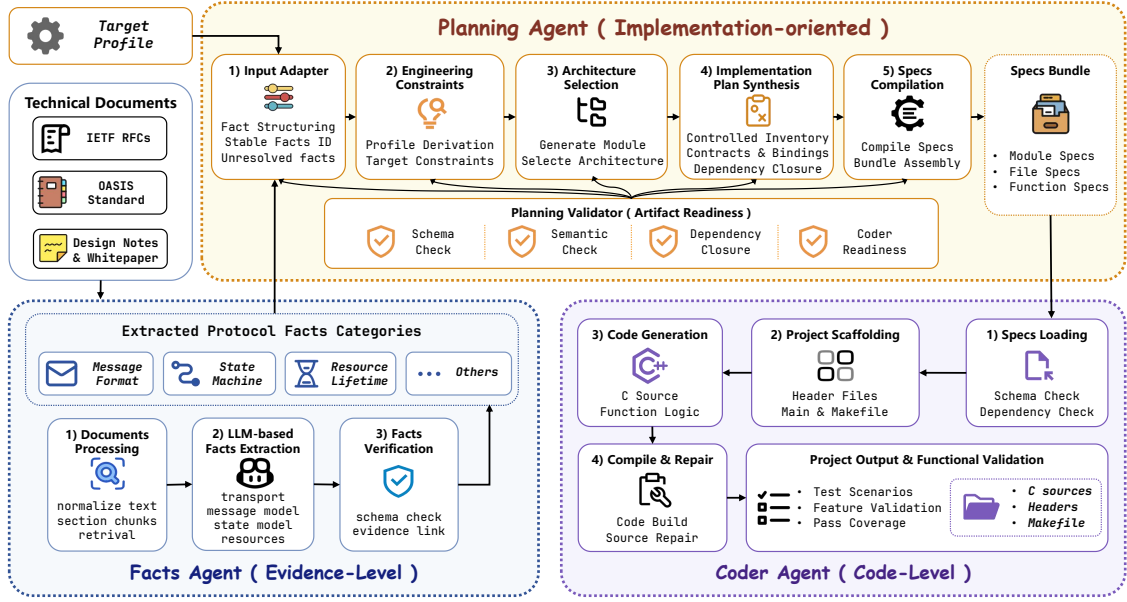


Figure 1: Overview

Prior work also demonstrates the value of explicit specifications. Narcissus and EverParse synthesize verified parsers and serializers from binary-format specifications (Delaware et al. 2018; Ramananandro et al. 2019), but assume formal inputs and focus on encoding and decoding. SpecGen generates formal specifications through mutation and validation (Ma et al. 2025); SYSSPEC uses multipart functional, module, and concurrency specifications for file-system generation (Liu et al. 2026); and Rubik uses domain-specific abstractions and compilation constraints for middlebox stacks (Li et al. 2021). These systems do not derive project-wide engineering specifications from factual knowledge in protocol documents.

2.3 The Missing Layer: Protocol Planning

A critical gap therefore remains between protocol fact extraction and specification-guided code generation. The former provides structured protocol knowledge primarily for analysis and verification, whereas the latter generates code from explicit specifications but commonly assumes that these specifications have been manually provided. For protocol implementation generation, the central problem is how to transform evidence-grounded protocol facts into implementation-oriented engineering specifications. Existing studies have begun to explore this direction. SAGE combines protocol ambiguity resolution with code generation, demonstrating the feasibility of generating implementations from specification text (Yen et al. 2021). APG constructs a pipeline spanning RFC analysis, code generation, and code review, and evaluates it across multiple protocols (Long, Su, and Han 2026), it can generate only a partial protocol implementation. However, these approaches generally treat intermediate representations as internal mechanisms rather than explic-

itly modeling and systematically defining the transformation from protocol facts to engineering specifications.

3 SpecForge Design

This section defines the intermediate planning task and introduces SpecForge, a modular framework for constructing coder-usable implementation-oriented specifications. As shown in Fig.1, the Facts Agent extracts protocol facts, the Planning Agent transforms them into engineering specifications, and the Coder Agent consumes these specifications to generate code.

3.1 Problem Formulation

Given a protocol technical document $\mathcal{D}$, we extract a set of **Protocol Facts** $\{F = f_1, \dots, f_N\}$, where each fact $f_i = (c_i, x_i, e_i)$ consists of a fact category, structured content, and supporting evidence from the document. Given the target implementation constraints $\mathcal{G} = (r, l, \mathcal{C})$, where r , l , and $\mathcal{C}$ denote the target role, implementation language, and functional scope, respectively, protocol planning aims to generate **Implementation-oriented Specification** $\mathcal{S}$ for downstream code generation, where $\mathcal{S}$ describes the structural, interface, and behavioral constraints. We formalize this task as the following constrained transformation: $\Pi : (\mathcal{F}, \mathcal{G}) \mapsto \mathcal{S}$, and $\mathcal{S}$ should satisfy $\text{Valid}(\mathcal{S} \mid \mathcal{F}, \mathcal{G}) = \text{Preserve}(\mathcal{S}, \mathcal{F}, \mathcal{G}) \wedge \text{Grounded}(\mathcal{S}, \mathcal{F}) \wedge \text{Consistent}(\mathcal{S}) \wedge \text{Usable}(\mathcal{S})$. Here, Preserve requires $\mathcal{S}$ to preserve the key protocol semantics within the target scope; Grounded requires its contents to be supported by the input facts; Consistent requires the absence of cross-module or cross-layer conflicts within the specification; and Usable requires the specification to be directly consumable by the downstream code-generation process.

3.2 Facts Agent

The goal of the Facts Agent is not to generate a comprehensive protocol summary, but to construct a fact base with a well-defined scope, traceable evidence, and suitable structure for engineering derivation. Protocol rules are distributed across different sections of technical documents, whereas the target role and functional scope are implementation requirements. Directly mixing the two can cause scope instructions to be misinterpreted as normative protocol requirements. SpecForge therefore uses the target profile only to determine the protocol surface to be covered. It then performs targeted retrieval around relevant protocol entities and extracts facts such as message, state, and interaction rules, while checking whether cross-category references are closed. The resulting structured protocol facts explicitly distinguish documented facts, scope decisions, and open questions, and retain documentary evidence for each normative fact. The Planning Agent receives only this fact file, thereby separating protocol semantics from implementation scope.

3.3 Planning Agent

Protocol facts primarily describe externally observable behavior rather than how software should be organized. Consequently, the same set of facts may admit multiple reasonable implementations, and planning is neither a simple extraction task nor a process that can be completed reliably through a single generation step. SpecForge models this process as evidence-constrained progressive commitment: the LLM proposes localized designs that require semantic judgment, while deterministic mechanisms progressively fix design identities, propagate constraints, and validate global consistency.

To incorporate protocol-specific knowledge, the planning process first identifies protocol characteristics that affect implementation structure and activates general network engineering rules accordingly. For example, stream-based transport requires explicit handling of message boundaries and connection lifecycles, while stateful interactions require explicit state management and rejection of invalid transitions. These rules specify only the engineering constraints that an implementation must satisfy, without prescribing protocol-specific structures. This design converts network engineering knowledge that would otherwise depend on the model’s implicit experience into explicit constraints, preventing general-purpose software planning from omitting mechanisms essential to protocol implementation.

SpecForge then progressively narrows the decision space through multi-stage planning. To suppress cross-stage drift, such as symbol renaming, duplicate types, interface boundary violations, and undefined references, the early stages establish the implementation scope, architectural boundaries, and artifact ownership. Subsequent stages may only extend registered entities with types, interfaces, behaviors, and call contracts, rather than arbitrarily modifying their identities or ownership. For large local derivations, the system performs generation, binding, validation, and commitment as partitioned transactions; a failure rolls back only the current partition. A final implementability gate checks type and interface

closure, resource lifecycles, call dependencies, and runtime paths. Information that can be uniquely determined is completed deterministically, while unresolved issues are handled by the LLM through localized patches within a constrained context. Only results satisfying structural consistency, semantic closure, and coder-loadability requirements are released. Here, *Preserve* requires coverage of all in-scope protocol obligations, *Grounded* requires fact-level provenance for normative constraints, and *Consistent* requires unique ownership and closed cross-artifact references. SpecForge thus transforms open-ended software design into a set of decision transactions with explicit permissions, evidence sources, and commitment conditions.

3.4 Protocol-Aware Specifications and Coder

General-purpose software specifications can describe module structure, but they are insufficient to constrain protocol semantics that propagate across files. To prevent different modules from implementing locally reasonable yet globally inconsistent data models, SpecForge uses module-level specifications to establish the protocol scope, module responsibilities, and public boundaries, ensuring that subsequent code generation shares a common architectural boundary. File-level specifications further define type ownership, public interfaces, and access constraints to unify data representations, resource ownership, and compilation boundaries. Function-level specifications bridge the gap between local code logic and end-to-end protocol behavior. Beyond function signatures, each *FUNCTION_SPEC* encodes protocol behavior and cross-function contracts, ensuring that protocol fields map to consistent data representations, protocol events trigger the correct states and responses, and resources are consistently propagated and released along call paths. The three specification levels therefore constrain architecture, representation, and behavior, respectively, allowing independently generated code components to preserve consistent protocol semantics.

With these specifications, the Coder Agent does not need to infer the system design again. Public headers and build files are generated deterministically from the specifications, while the LLM generates only source files under a fixed ABI. Before generation, the specification loader checks specification closure and verifies that the public headers are compilable. *Usable* requires a schema-valid, dependency-closed specification bundle that the Coder can load and compile into public headers. Subsequent repair handles only localized source-file defects, while public-interface and cross-module contract errors are reported back to the planning stage. Compilation and behavioral validation therefore assess whether the specifications are sufficient to support implementation, rather than completing missing design decisions during code generation.

4 Experiment Setup

4.1 Evaluation Scope and Protocols

We evaluate two aspects of SpecForge: how different planning methods affect the structural and semantic quality of generated protocol code, and how protocol-aware specification content affects downstream C generation. The study

covers an MQTT planning comparison and controlled gold-specification ablations across MQTT, CoAP, SMTP, and HTTP/1.1. All tasks generate modular C projects within predefined minimal functional scopes. Protocol roles, scopes, and tests are summarized in Appendix A; these tests do not represent full RFC compliance.

4.2 Experimental Configurations and Procedure

We design the following controlled experiments around two research questions.

For RQ1, we compare four generation pipelines on MQTT: **M0**: FS-Direct-Coder codes directly from protocol facts, **M1**: NL-Plan-Code codes after natural-language planning, **M2**: One-shot Structured Planning generates the complete structured specifications at once, and **M3**: Full-SpecForge constructs the same specifications through staged planning and validation. All settings share the protocol facts, model, implementation tasks, scaffolding, repair budget, and tests, but M0/M1 use generic bounded repair whereas M2/M3 use SpecForge-specific repair; M2 and M3 additionally share the final schema and downstream Coder, differing only in one-shot versus validator-constrained staged construction.

For RQ2, we isolate the effect of specification representation across the four protocols. For each protocol, the **S4: Full Protocol-Aware** specification is manually extracted from an existing, manually implemented reference implementation verified to run correctly. **S1–S3** are nested variants manually pruned from the same S4 artifact following the SpecFS format: **S1** retains only local specifications; **S2** adds module and file topology; and **S3** further retains public types and structured interfaces. All levels share the same implementation scope, underlying engineering design, implementation tasks, and validation procedure, and differ only in the specification information visible to the code generator.

All configurations use qwen3-max-2026-01-23 and are independently executed 10 times on Ubuntu 22.04. After generation, each project is compiled. If compilation fails, RQ1 M0/M1 use generic bounded repair, whereas RQ1 M2/M3 and all RQ2 settings use SpecForge-specific repair; repair stops when compilation succeeds, the budget is exhausted, or a blocking error is encountered. Successfully compiled projects are then subjected to protocol behavioral tests. Behavioral test failures do not trigger further repair.

4.3 Evaluation Metrics

RQ1 Metrics. For RQ1, we first report the **code-bearing completion rate**, i.e., the number of planning runs that produce a specification bundle accepted by the downstream Coder and consequently yield code for analysis. For code-bearing runs, we report the **number of affected projects** and the **number of root-cause defect groups** in each C1–C5 family. The former denotes how many projects contain a family before repair; a single project may belong to multiple families. The latter is obtained by manually reviewing and correcting the C1–C5 assignments, with duplicate or cascading diagnostics caused by the same underlying defect merged into one group. We also measure type visibility, clean translation units, and semantic breadth before repair. Because M2 yields only one code-bearing run, its code-level

measurements are descriptive and are not compared as if they represented ten completed runs. The five root-cause families are local C/build, header and visibility, API contract, type and ownership, and protocol model.

RQ2 Metrics. RQ2 evaluates implementation quality, repair cost, and generation overhead. Implementation quality includes the **compilation success rate**, the **number of behavioral tests passed**, and the **average number of compilation errors** before and after repair. Repair cost includes the **average number of repair iterations** and the **average number of modified files per repair iteration**. Generation overhead includes the **average lines of C code** and the **average token consumption**. For runs that fail to compile or do not execute behavioral tests, all corresponding behavioral scenarios are counted as failed.

5 Evaluation

We separately evaluate how planning methods affect generated protocol code quality and how protocol-aware specifications affect downstream generation. RQ1 compares code-bearing completion and the pre-repair structural and semantic quality of generated MQTT projects, while RQ2 uses manually constructed gold specifications to isolate effects on buildability, bounded repair, and protocol behavior. Both studies cover predefined core functionality rather than full protocol compliance.

5.1 Planning Utility Evaluation Results

Direct generation and natural-language planning trade semantic breadth for structural consistency. As shown in Table 1, M0 and M1 both produce code in all ten runs. M1 increases the number of runs meeting the semantic-breadth threshold from 2/10 under M0 to 4/10, but its pre-repair C1–C5 root-cause groups also rise from 97 to 142. The difference is driven mainly by type and ownership root causes: C4 affects 9/10 M1 projects and contributes 30 groups, compared with 5/10 projects and 13 groups for M0. M1 also has more header and visibility groups than M0 (26 versus 6). Together with their similar type visibility (50.9% versus 50.0%) and clean-translation-unit rates (46.4% versus 50.0%), these results show that unconstrained natural-language planning can cover more protocol obligations without reliably coordinating cross-file types, ownership decisions, and declarations.

Structured, staged planning improves both completion reliability and pre-repair quality. M2 produces code in only 1/10 planning runs; that run has one root-cause group, 64.5% type visibility, and 90.0% clean translation units, but does not meet the semantic-breadth threshold. These single-run measurements are therefore descriptive only. By contrast, M3 produces code in 10/10 runs and records 37 pre-repair root-cause groups, 61.9% fewer than M0 and 73.9% fewer than M1. M3 achieves 77.0% type visibility and meets the semantic-breadth threshold in 9/10 runs, compared with 2/10 for M0, 4/10 for M1, and 0/1 for the sole code-bearing M2 run. Its clean-translation-unit rate of 80.0% is higher than M0 and M1, although lower than M2’s single-run value. Because M2 and M3 use the same final schema and downstream pipeline, their 1/10 versus 10/10 completion gap pro-

Table 1: C1–C5 root-cause defect groups across planning methods.

Cat.	Root-cause family	M0		M1		M2		M3	
		Proj.	Groups	Proj.	Groups	Proj.	Groups	Proj.	Groups
C1	Local C/build	9/10	20 → 3	8/10	18 → 4	0/1	0 → 0	3/10	5 → 5
C2	Header and visibility	4/10	6 → 5	8/10	26 → 12	1/1	1 → 1	7/10	10 → 8
C3	API contract	10/10	48 → 40	10/10	51 → 47	0/1	0 → 0	5/10	12 → 13
C4	Type and ownership	5/10	13 → 12	9/10	30 → 29	0/1	0 → 0	3/10	7 → 2
C5	Protocol model	6/10	10 → 8	7/10	17 → 17	0/1	0 → 0	2/10	3 → 1
Total groups (C1–C5)		97 → 68		142 → 109		1 → 1		37 → 29	

Note: M0–M3 are defined in Sec. 4.2; arrows show pre→post groups; *Proj.* is measured pre-repair.

Table 2: Pre-repair structural and semantic quality.

Metric	M0	M1	M2	M3
Type visibility	50.0% [48.0, 52.4] 116/243	50.9% [47.1, 53.0] 125/249	64.5% [64.5, 64.5] 20/31	77.0% [73.6, 77.9] 284/373
Clean translation units	50.0% [37.5, 55.4] 39/79	46.4% [38.8, 55.4] 33/73	90.0% [90.0, 90.0] 9/10	80.0% [73.0, 85.1] 51/64
Semantic breadth $\geq 3/5$	2/10 (20.0%)	4/10 (40.0%)	0/1 (0.0%)	9/10 (90.0%)

Note: The first two rows show median [IQR] and pooled passed/total (second line); metric definitions are in Sec. 4.3. All values are pre-repair.

vides direct evidence for the reliability gained from staged, validator-constrained specification construction.

Repair reduces residual root-cause groups but does not erase planning-stage differences. After repair, root-cause groups decrease from 97 to 68 for M0, from 142 to 109 for M1, remain at 1 for the sole M2 project, and decrease from 37 to 29 for M3. M0, M1, and M3 remove 29, 33, and 8 groups, respectively; M3 nevertheless retains the fewest residual groups among the three methods with ten code-bearing runs. Within M3, type and ownership groups decrease from 7 to 2 and protocol model groups from 3 to 1, whereas API contract groups increase from 12 to 13. Repair therefore complements planning but cannot replace coherent project-wide decisions established before code generation.

5.2 Specification Ablation Results

Protocol-aware specifications achieve the highest buildability. Across 40 runs over four protocols, all S4 projects compiled successfully within the bounded repair budget (see Table 3), yielding an overall compilation success rate of 100%, compared with 50.0%, 77.5%, and 92.5% for S1, S2, and S3, respectively. The difference is most pronounced for MQTT, where the compilation success rate increases from 0/10 under S1, 1/10 under S2, and 7/10 under S3 to 10/10 under S4. These results indicate that local function specifications, project topology, and structured interfaces progressively improve buildability, but remain insufficient to reliably coordinate shared types, symbol visibility, and cross-file contracts in multi-module protocol implementations. Table 4 further shows that the downstream utility of the same S4 gold specification extends across multiple code-generation models.

Protocol-aware specifications improve behavioral correctness. The overall behavioral test pass rates for the four specification levels are 27.9% (81/290), 46.6% (135/290),

70.3% (204/290), and 94.8% (275/290), respectively. For SMTP, all four settings achieve a compilation success rate of 10/10, yet S1–S3 pass only 15–30 behavioral scenarios, whereas S4 passes 70/80. HTTP and MQTT exhibit the same trend. This suggests that file structure and ABI consistency mainly ensure that the generated code is buildable, while correct state transitions, message processing, and error paths require explicit behavioral contracts, wire-format mappings, and negative constraints. CoAP passes 60/60 scenarios under all settings, indicating that local specifications may already be sufficient for weakly stateful protocols with short interaction flows. However, this ceiling effect does not generalize to more complex protocols.

The defect distribution reveals the limitations of SpecFS. Under S1, type and layout defects affect 82.5% of the runs, while symbol and visibility defects affect 50.0%. After adding project topology and structured interfaces, these two categories still affect 50.0% and approximately 50% of the runs under S2 and S3, respectively (see Figure 2). In contrast, S4 reduces type and layout defects to 22.5% and completely eliminates field/schema and API/signature defects. The remaining issues primarily involve symbol visibility and local syntactic expressions rather than cross-module interface or protocol data-model inconsistencies. These results indicate that the protocol-specific extensions in SpecForge unify shared data models, interfaces, call relationships, and protocol field constraints within a single specification, thereby reducing cross-module drift at its source.

The quality gains are not driven by higher generation cost. The generated C code sizes remain broadly comparable across specification levels, and S4 does not consistently incur the highest token consumption. For MQTT, S4 consumes an average of 101,510.4 tokens, fewer than S2 and S3; for SMTP, its token consumption is also lower than that of S3. Meanwhile, S4 reduces the average number of MQTT repair

Table 3: Spec-ablation results across four protocols.

Protocol	Setting	Verification		Compile errors		Repair		Generation	
		Compile	Behavior	Init.	Final	Rounds	Files	C LoC	Tokens
HTTP	S1: Local	0/10	0/90	19.30	5.00	1.50	2.70	1,535.00	67,129.00
	S2: Project graph	10/10	54/90	27.40	0.00	1.20	3.00	1,836.10	91,692.80
	S3: Interfaces	10/10	72/90	26.70	0.00	1.00	3.00	1,581.80	109,359.60
	S4: Protocol-aware	10/10	90/90	22.20	0.00	1.00	2.90	1,678.30	117,192.80
MQTT	S1: Local	0/10	0/60	35.00	8.60	2.70	4.70	2,179.60	114,253.70
	S2: Project graph	1/10	6/60	22.20	8.30	2.80	2.40	2,120.90	158,050.90
	S3: Interfaces	7/10	42/60	8.90	0.30	1.70	2.30	2,065.00	144,944.40
	S4: Protocol-aware	10/10	55/60	8.40	0.00	1.00	1.40	2,227.70	101,510.40
CoAP	S1: Local	10/10	60/60	10.50	0.00	0.90	1.30	1,308.30	59,397.20
	S2: Project graph	10/10	60/60	1.50	0.00	0.50	0.50	1,298.50	61,304.50
	S3: Interfaces	10/10	60/60	0.00	0.00	0.00	0.00	1,280.20	75,637.70
	S4: Protocol-aware	10/10	60/60	2.60	0.00	0.90	1.00	1,281.60	89,232.60
SMTP	S1: Local	10/10	21/80	29.20	0.00	1.00	3.40	1,970.80	98,587.80
	S2: Project graph	10/10	15/80	30.60	0.00	1.00	3.90	1,980.00	134,439.40
	S3: Interfaces	10/10	30/80	29.70	0.00	1.00	3.40	1,918.50	150,209.40
	S4: Protocol-aware	10/10	70/80	8.40	0.00	1.00	2.50	2,008.50	135,083.70

Notes. S1–S4 denote SpecFS-Local, SpecFS-ProjectGraph, SpecFS-InterfaceGrounded, and Full Protocol-Aware, respectively. Compile and behavior are passed/total; all other entries are means.

Table 4: Cross-model MQTT results with the S4 specification.

Model	Compile	Init. errors	Tokens	Behavior
Qwen3-Max	10/10	8.40	101,510	55/60
DeepSeek-V4	5/10	4.30	139,208	28/60
GPT-5.4	10/10	0.00	93,191	60/60

Notes. Compile/behavior are passed/total; errors/tokens are means; token counts are rounded.

iterations to 1.0 and the average number of modified files per iteration to 1.4, while decreasing the initial number of SMTP compilation errors from approximately 29–31 under the first three settings to 8.4. Therefore, the benefits of S4 arise primarily from the completeness and organization of the specification information rather than from simply increasing the generation context or code volume.

Protocol-specific constraints are critical for complex implementations. General-purpose SpecFS specifications can express local implementation tasks, while module topology and structured interfaces improve project buildability. However, reliable protocol behavior still depends on cross-function behavioral contracts, state semantics, wire-format bindings, call constraints, and negative constraints. By extending the specification along these protocol-specific dimensions, SpecForge simultaneously improves compilation stability, behavioral correctness, and repair locality, particularly for protocols with complex states and interactions.

6 Conclusion

This paper presents SpecForge, which formalizes implementation-oriented specification planning and derives protocol-aware specifications through staged validation.

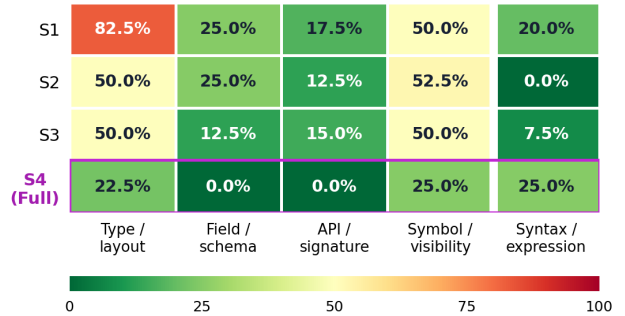


Figure 2: **Initial Compile Diagnostic Taxonomy.** Each cell reports the percentage of runs affected by the corresponding diagnostic category across four protocols.

SpecForge consolidates project-wide engineering information to mitigate cross-module inconsistencies and extends general-purpose specifications with protocol-specific constraints. On MQTT, SpecForge reduces pre-repair root-cause groups by 61.9% and 73.9%, raises median type visibility from 50.0% and 50.9% to 77.0%, and increases runs meeting the semantic-breadth threshold from 2/10 and 4/10 to 9/10; it also retains the fewest post-repair groups. Across HTTP/1.1, MQTT, CoAP, and SMTP, protocol-aware specifications enable all 40 generation runs to compile successfully and improve the behavioral test pass rate from 70.3% with general-purpose code specifications to 94.8%. These results show that staged validation improves protocol-aware specification construction, while protocol-aware specification content is critical for downstream buildability and behavioral correctness. Future work will improve planning stability and advance behavior-complete protocol implementation.

References

- Bairi, R.; Sonwane, A.; Kanade, A.; C, V. D.; Iyer, A.; Parthasarathy, S.; Rajamani, S.; Ashok, B.; and Shet, S. 2024. Codeplan: Repository-level coding using llms and planning. *Proceedings of the ACM on Software Engineering*, 1(FSE): 675–698.
- Delaware, B.; Suriyakarn, S.; Ye, Q.; Chlipala, A.; et al. 2018. Narcissus: Deriving Correct-By-Construction Decoders and Encoders from Binary Formats. *arXiv preprint arXiv:1803.04870*.
- Hong, S.; Zhuge, M.; Chen, J.; Zheng, X.; Cheng, Y.; Wang, J.; Zhang, C.; Yau, S.; Lin, Z.; Zhou, L.; et al. 2024. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, volume 2024, 23247–23275.
- Ishtiaq, A. A.; Das, S. S. S.; Rashid, S. M. M.; Ranjbar, A.; Tu, K.; Wu, T.; Song, Z.; Wang, W.; Akon, M.; Zhang, R.; and Hussain, S. R. 2024. Hermes: Unlocking Security Analysis of Cellular Network Protocols by Synthesizing Finite State Machines from Natural Language Specifications. In *33rd USENIX Symposium on Security and Privacy (USENIX Security 24)*, 4445–4462. Philadelphia, PA: USENIX Association. ISBN 978-1-939133-44-1.
- Jiang, J.; Wang, F.; Shen, J.; Kim, S.; and Kim, S. H. 2026. A Survey on Large Language Models for Code Generation. *ACM Transactions on Software Engineering and Methodology*, 35(2): 1–72.
- Jin, H.; Huang, L.; Cai, H.; Yan, J.; Li, B.; and Chen, H. 2024. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current Challenges and Future. *arXiv preprint arXiv:2408.02479*.
- Li, H.; Wu, C.; Sun, G.; Zhang, P.; Shan, D.; Pan, T.; and Hu, C. 2021. Programming Network Stack for Middleboxes with Rubik. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, 551–570. USENIX Association. ISBN 978-1-939133-21-2.
- Liu, Q.; Zou, M.; Zhang, H.; Du, D.; Xia, Y.; and Chen, H. 2026. Sharpen the Spec, Cut the Code: A Case for Generative File System with SYSSPEC. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*, 291–311. Santa Clara, CA: USENIX Association. ISBN 978-1-939133-53-3.
- Long, J.; Su, J.; and Han, B. 2026. LLMs Unleashed: Generating Protocol Code from RFC Specifications. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(1): 809–817.
- Ma, L.; Liu, S.; Li, Y.; Xie, X.; and Bu, L. 2025. SpecGen: Automated Generation of Formal Program Specifications via Large Language Models. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 16–28. IEEE.
- Ma, X.; Luo, L.; and Zeng, Q. 2024. From One Thousand Pages of Specification to Unveiling Hidden Bugs: Large Language Model Assisted Fuzzing of Matter IoT Devices. In *33rd USENIX Security Symposium (USENIX Security 24)*, 4783–4800. Philadelphia, PA: USENIX Association. ISBN 978-1-939133-44-1.
- Meng, R.; Mirchev, M.; Böhme, M.; and Roychoudhury, A. 2024. Large Language Model Guided Protocol Fuzzing. In *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS 2024)*. Internet Society.
- Pacheco, M. L.; von Hippel, M.; Weintraub, B.; Goldwasser, D.; and Nita-Rotaru, C. 2022. Automated Attack Synthesis by Extracting Finite State Machines from Protocol Specification Documents. In *2022 IEEE Symposium on Security and Privacy (SP)*, 51–68. IEEE.
- Qian, C.; Liu, W.; Liu, H.; Chen, N.; Dang, Y.; Li, J.; Yang, C.; Chen, W.; Su, Y.; Cong, X.; et al. 2024. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, 15174–15186.
- Ramananandro, T.; Delignat-Lavaud, A.; Fournet, C.; Swamy, N.; Chajed, T.; Kobeissi, N.; and Protzenko, J. 2019. {EverParse}: Verified secure {Zero-Copy} parsers for authenticated message formats. In *28th USENIX Security Symposium (USENIX Security 19)*, 1465–1482.
- Sharma, P.; and Yegneswaran, V. 2023. PROSPER: Extracting Protocol Specifications Using Large Language Models. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*, 41–47. New York, NY, USA: Association for Computing Machinery.
- Wang, F.; Wu, J.; Nan, Y.; Aafer, Y.; Zhang, X.; Xu, D.; and Payer, M. 2022. ProFactory: Improving IoT Security via Formalized Protocol Customization. In *31st USENIX Security Symposium (USENIX Security 22)*, 3879–3896. Boston, MA: USENIX Association. ISBN 978-1-939133-31-1.
- Yen, J.; Lévai, T.; Ye, Q.; Ren, X.; Govindan, R.; and Raghavan, B. 2021. Semi-Automated Protocol Disambiguation and Code Generation. In *Proceedings of the ACM SIGCOMM 2021 Conference*, 272–286. New York, NY, USA: Association for Computing Machinery.
- Zheng, M.; Xie, D.; and Zhang, X. 2025. Large Language Models for Validating Network Protocol Parsers. In *2025 IEEE Security and Privacy Workshops (SPW)*, 56–64. IEEE.